1 INFORMATIONS GENERALES

Apprenti :	Nom: Tsybulevskyi	Prénom : Maksym
Lieu de travail :	ETML / VENNES / 1010 LAUSANNE	
Chef de projet :	Nom: Carrel	Prénom: Xavier
	Du vendredi 06.11.2026 au vendredi 18.12.2026	
Dates de réalisation :	5 périodes par semaine, 7 semaines (moins les jours fériés !)	
Horaire de travail :	35 périodes	
Temps total :	26 heures 15 min	

2 PROCÉDURE

- Tous les apprentis réalisent le projet sur la base d'un cahier des charges.
- Le cahier des charges est présenté, commenté et discuté en classe.
- Les apprentis sont entièrement responsables de la sécurité et sauvegarde de leurs données.
- En cas de problèmes graves, les apprentis avertissent le client au plus vite.
- Les apprentis ont la possibilité d'obtenir de l'aide externe, mais ils doivent le mentionner.
- Les informations utiles à l'évaluation de ce projet sont disponibles au chapitre 8.

3 TITRE

Un ensemble d'exercices en **JavaScript moderne (ESM)** avec **tests jest.**

4 SUJET

Réaliser une série d'exercices en JavaScript, écrits en **modules ESM** (export/import) et accompagnés de **tests unitaires automatisés avec jest**.

5 MATÉRIEL ET LOGICIEL À DISPOSITION

- Un PC ETML
- Accès à Internet
- Visual Studio Code
- Node.js
- Git et github

6 PRÉREQUIS

Modules de programmation de base

7 CAHIER DES CHARGES

7.1 Description du projet

Ce projet contient une série de petits exercices en JavaScript moderne (ESM), conçus pour des débutants complets. Chaque exercice est accompagné de tests unitaires automatisés avec jest, pour valider que la solution fonctionne correctement.

7.2 Objectifs pédagogiques

- Apprendre les bases de JavaScript (fonctions, conditions, chaînes, etc.)
- Comprendre la syntaxe des modules ESM (export / import)
- Écrire du code testable
- Découvrir les tests unitaires avec jest

7.3 Technologies Utilisées

Node.js est un environnement d'exécution JavaScript basé sur le moteur V8 de Chrome, qui permet d'exécuter du code JavaScript côté serveur.

Il est particulièrement utilisé pour créer des applications web rapides, évolutives et non bloquantes grâce à son modèle asynchrone et orienté événements.

npm (Node Package Manager) est le gestionnaire de paquets de **Node.js**. Il permet aux développeurs d'installer, de partager et de gérer facilement des librairies et des outils JavaScript. Voir la plateforme en ligne (npmjs.com).

npm est inclus avec l'installation de Node.js. Cela signifie que lorsque vous installez **Node.js** sur votre machine, **npm** est automatiquement installé en même temps.

Jest est un framework de tests JavaScript développé par Facebook, principalement utilisé pour tester des applications React, mais compatible avec tout projet JavaScript. Il offre une configuration minimale, une exécution rapide des tests, et des fonctionnalités intégrées comme les mocks, les snapshots et la surveillance des changements de fichiers.

ECMAScript Modules (ESM)

Le projet est structuré en utilisant la syntaxe ESM, permettant d'importer et d'exporter des fonctions et des variables entre les différents fichiers JavaScript. Cela permet de séparer les différentes responsabilités du code et de maintenir une structure propre et modulable.

Votre code devra être documenté avec **JSDoc**, un standard de documentation pour JavaScript. **JSDoc** permet de générer automatiquement une documentation à partir des commentaires du code, ce qui améliore la lisibilité et la maintenabilité du projet.

Le recours à l'intelligence artificielle est strictement interdit dans ce projet

7.4 Organisation

Votre enseignant vous met à disposition un code de départ pour l'ensemble des exercices.

Les exercices sont organisés autour de 5 axes :

```
exercices/
├── axe-fondamentaux/
├── axe-controle/
├── axe-fonctions/
├── axe-donnees/
└── axe-async/
```

Chaque axe contient plusieurs missions/exercices.

Nous pouvons voir ces 5 axes d'une autre manière :



7.5 Installation

Dézippez le code de départ reçu.

Avant de commencer à réaliser les exercices, vous devez installer les dépendances javascript avec la commande :

```
npm install
```

Rendez-vous sur Bullejs et :

1. Connectez-vous. Vos credentials sont :
   - Utilisateur = votre prénom (avec la première lettre de votre nom si vous êtes plusieurs à porter le même prénom)
   - Mot de passe = votre prénom
2. Allez sur votre profil et changez votre mot de passe

7.6 Création d'un dépôt github

Vous devez sauvegarder votre travail dans un dépôt github.

Conseil: vos fichiers doivent être sur le disque fixe (p.ex: Documents/Github). Ne travaillez ***PAS*** sur votre SSD.

Une fois que votre repo est créé et publiez, allez mettre son URL :

- Dans vos paramètres sur Bullejs
- Dans Marketplace

7.7 Documentation

Vous avez une question sur la syntaxe javascript ? Vous vous demandez comment réalise-t-on des boucles en javascript ? Ou encore mais qu'est- ce qu'une promesse ?

Organisez une documentation locale. Lors des différentes évaluations, vous aurez droit à cette documentation. Autant apprendre à l'utiliser pendant les exercices.

Nous vous conseillons fortement la documentation javascript.info, dont vous pouvez télécharger le ZIP.

Il faut le télécharger maintenant parce que l'accès internet sera coupé durant les évaluations.

Nous vous recommandons de la stocker dans votre repo local pour que vous ayez tout au même endroit. Mais attention : comme il s'agit d'un contenu téléchargé qui n'est pas le fruit de votre travail, il faut l'exclure des éléments suivis par votre repo dans le **`.gitignore`**.

7.8 Exécuter les tests

Pour exécuter les tests unitaires automatisés avec jest :

Soit exécuter tous les tests avec la commande :

```
npm run test
```

Soit exécuter tous les tests d'une mission avec la commande :

```
npm run test -- exercices/axe-controle/portes-citadelle/
```

7.9 Réaliser les exercices

Pour chaque mission/exercice vous devez réaliser le code mais également la JSDoc !

Pour chaque exercice :

- Signalez le début d'activité avec "Prendre" dans Bullejs.
- Complétez la fonction dans son fichier dans votre repo local. Une fois modifiée, lancez les tests et corrigez si nécessaire jusqu'à ce que tout fonctionne.
- Faites commit / push
  - Veillez à bien nommer votre commit (voir les conventions techniques au besoin)
  - Dans la première ligne de la description, indiquez entre [ ] le temps *en minutes* qu'il vous a fallu pour faire le contenu du commit, suivi du status [WIP] ou [DONE]
- Signalez la fin de l'exercice avec le bouton "Fini". Saisissez l'URL du commit que vous venez de push
- Choisissez et démarrez un nouvel exercice sans plus attendre

Bonne chance et bon apprentissage ! 💡

7.9.1 Amélioration continue par les tests

Le passage réussi des tests ne marque pas la fin d'un exercice, mais le début d'une phase d'amélioration. Une fois que tous les tests jest passent, vous devez utiliser ces tests comme un **outil de confiance** pour **refactoriser**, **simplifier** et **optimiser** votre code sans craindre de casser le comportement attendu. L'objectif n'est pas seulement d'obtenir un « vert » dans jest, mais de comprendre comment les tests soutiennent la **qualité**, la **lisibilité** et la **maintenabilité** du code. Les tests servent donc de filet de sécurité pour expérimenter, clarifier les intentions du code et apprendre à itérer proprement.

7.10 Utiliser un Debugger

Pour bien développer, il faut de bons outils. Parmi ceux-ci, il y a naturellement le debugger ! Il est attendu de vous que vous sachiez vous en servir.

Voici comment l'utiliser :

- Dans VS Code, ouvrez la palette de commandes avec : Ctrl + Shift + P
- Recherchez puis sélectionnez : **JavaScript Debug Terminal** ou, selon la langue de VS Code **Terminal de débogage JavaScript**
  Un nouveau terminal de débogage s'ouvre.
- Placez un point d'arrêt dans votre fichier .js en cliquant dans la marge à gauche du numéro de ligne.
- Lancez les tests de la mission depuis ce terminal.
  Par exemple :

```
npm run test -- exercices/axe-controle/portes-citadelle/
```
Vous pouvez également lancer tous les tests :
```
npm test
```
Lorsque l'exécution atteint votre point d'arrêt, VS Code suspend le programme.

Bon débogage !

7.11 Travail et documents à rendre à la fin du projet

A la fin de ce projet vous devez nous rendre via une **release Github** dans votre dépôt, à laquelle vous joignez (asset) votre journal de travail en format PDF.

Remarque : Un rapport n'est pas demandé afin que vous puissiez vous concentrer sur l'apprentissage du JavaScript.

8 Évaluation

1. Plusieurs évaluations auront lieu pendant le projet sous forme d'exercices similaires à ceux réalisés pendant le projet permettant d'évaluer le critère « Maîtrise technique » de la grille d'évaluation
2. Votre enseignant peut effectuer une ou plusieurs évaluations impromptues à l'oral tout au long du projet.
3. Vous devez être en capacité d'expliquer à votre enseignant chaque ligne de code que vous avez écrites.
4. Chaque élève possède **4 jokers** au départ du projet
5. Un joker est retiré à un élève suite à un comportement non professionnel (utilisation de l'IA, du téléphone, incapable d'expliquer le code fourni, etc)

L'évaluation est faite dans Marketplace. Les bulles d'informations ( ) des critères vous disent comment chaque critère est évalué dans ce projet.